

Инструкция и образец

ПП.04.01 «Сопровождение и обслуживание программного обеспечения компьютерных систем»

Сборка приложения C# WPF, затем подключение CSV · 30 индивидуальных вариантов

09.02.07 Информационные системы и программирование · программист

ПМ.04 Сопровождение и обслуживание программного обеспечения компьютерных систем

Учебно-методический комплект

Сервисный журнал учебного центра

Отдельный пример: проектор и ноутбук, одна неполная запись принтера.

Устройство WPF-решения и запуск

Материалы для работы

[Скачать исходный проект ServiceDesk \(ZIP\)](#)

Начните с него: соберите и проверьте приложение без CSV.

Подготовка Visual Studio

На Windows распакуйте ServiceDesk-source.zip. В Visual Studio выберите «Открыть проект или решение» и укажите ServiceDesk.csproj. В обозревателе решений разверните App.xaml и MainWindow.xaml. Откройте свойства проекта или сам .csproj: TargetFramework — net10.0-windows, UseWPF — true. Для имеющейся версии .NET нужен установленный SDK; не меняйте целевую платформу без причины.

Роли файлов

App.xaml задаёт вход в WPF-приложение, App.xaml.cs создаёт главное окно. MainWindow.xaml описывает интерфейс: Grid из трёх строк, кнопки, DataGrid и TextBlock состояния. MainWindow.xaml.cs содержит модель Ticket, встроенные данные и обработчики нажатий. XAML не читает CSV; код окна не задаёт геометрию таблицы. Такое разделение помогает локализовать неисправность.

Первая проверка

В меню «Сборка» выберите «Собрать решение», затем F5. После открытия окна нажмите «Тестовые данные» и «Просрочено». Если окно пусто, проверьте Click у кнопки, вызов InitializeComponent и привязку TicketsGrid.ItemsSource. Если сборка не проходит, сначала прочитайте первую ошибку в списке ошибок и проверьте совпадение x:Class с namespace и именем класса.

Приложение до подключения CSV

Журнал заявок — проверка WPF-приложения

Тестовые данные Просрочено

Id	Объект	Ответственный	Срок	Статус
101	Проектор	Ирина	28.09.2026	Открыта
102	Ноутбук	Павел	29.09.2026	Завершена

Тестовых записей: 2

Реальная сборка стартового проекта: две встроенные записи в таблице; CSV ещё не используется.

Проверка расчёта на встроенных данных

Журнал заявок — проверка WPF-приложения

Тестовые данные Просрочено

Id	Объект	Ответственный	Срок	Статус
101	Проектор	Ирина	28.09.2026	Открыта
102	Ноутбук	Павел	29.09.2026	Завершена

Просрочено: 1

После нажатия «Просрочено» стартовое приложение показывает 1.

Как устроено окно и его события

Материалы для работы

[Скачать исходный проект ServiceDesk \(ZIP\)](#)

Начните с него: соберите и проверьте приложение без CSV.

Откройте конструктор

Дважды щёлкните MainWindow.xaml в обозревателе решений. Через вкладку XAML проверьте Window с `x:Class="ServiceDeskWpf.MainWindow"`. Внутри Grid первая строка Auto занимает высоту кнопок, средняя * отдаёт место таблице, нижняя Auto показывает результат. Чтобы добавить компонент, найдите его в панели элементов или внесите элемент в XAML в нужную строку Grid. После сохранения вернитесь в конструктор и проверьте расположение.

Кнопка и обработчик

В XAML у кнопки «Тестовые данные» указан `Click="LoadTestData_Click"`. Ctrl+щелчок по имени либо F12 ведёт в MainWindow.xaml.cs. Обработчик создаёт две Ticket, присваивает TicketsGrid.ItemsSource список и меняет StatusText.Text. Это связывает действие оператора с отображением данных. Проверка: после нажатия видны обе строки, после повторного нажатия число строк не удваивается.

Типы и формат даты

Ticket содержит Id, Item, Owner, Due и Status. В DataGrid колонки заданы явно, а Due форматируется как dd.MM.yyyy. Формат отображения не меняет DateTime в памяти; отдельный формат CSV задаётся в CsvStore.Write. Проверьте дату в таблице и в исходном файле отдельно.

Переход к CSV: сравнение проектов

Материалы для работы

[Скачать заготовку подключения CSV \(ZIP\)](#)

Откройте после первой проверки, чтобы изучить дефекты импорта и экспорта.

[Скачать демонстрационный CSV для проверки импорта](#)

Откройте до запуска, затем загрузите для проверки качества входных строк.

[Скачать демонстрационный CSV без дефектной строки \(demo-valid.csv\)](#)

Используйте для отдельной проверки экспорта и повторного импорта.

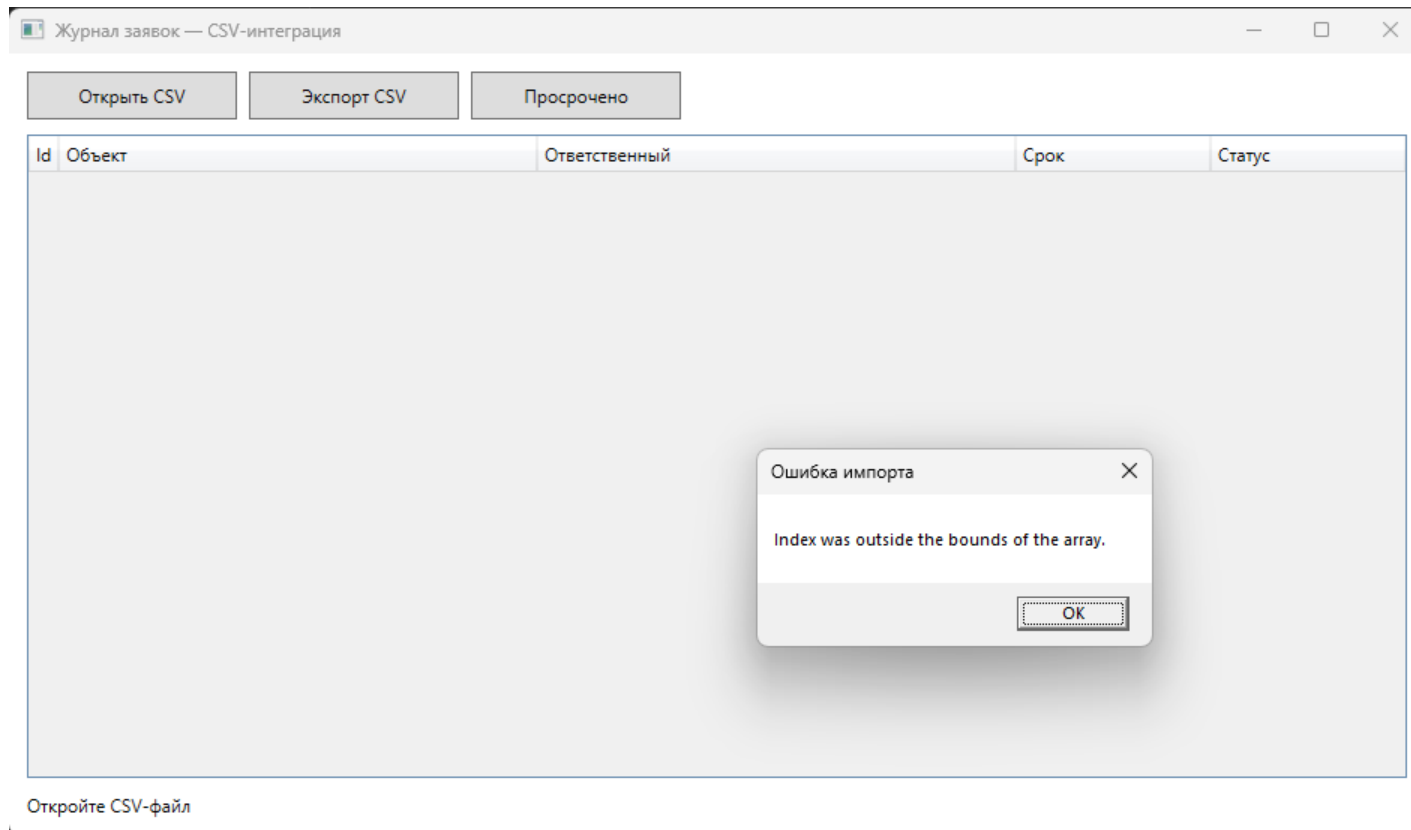
Рабочая копия

Сделайте копию проверенного стартового проекта. Скачайте ServiceDesk-csv-draft.zip и откройте его как отдельный проект. В Visual Studio через «Сравнить с» или обычный редактор сопоставьте MainWindow.xaml: появились две кнопки. В MainWindow.xaml.cs появились OpenCsv_Click, SaveCsv_Click и CsvStore. Добавляя элементы в собственную копию, следите за совпадением Click с именами методов; отсутствие обработчика даёт ошибку сборки.

Откуда берутся данные

Откройте demo.csv и demo-valid.csv рядом с окном. Кнопка «Открыть CSV» вызывает OpenFileDialog, путь переходит в CsvStore.Read, результат назначается TicketsGrid.ItemsSource. Не изменяйте demo.csv перед первым воспроизведением: иначе потеряется исходный симптом. Сначала запишите ожидаемые записи и строку ошибки.

Реальная ошибка первой интеграции



Черновая интеграция CSV, сборка .NET 10 на Windows: неполная строка 4 прерывает импорт, окно остаётся пустым.

Диагностика D1: путь строки через код

Материалы для работы

[Скачать заготовку подключения CSV \(ZIP\)](#)

Откройте после первой проверки, чтобы изучить дефекты импорта и экспорта.

[Скачать демонстрационный CSV для проверки импорта](#)

Откройте до запуска, затем загрузите для проверки качества входных строк.

[Скачать исправленный демонстрационный проект \(ZIP\)](#)

Сверяйтесь после самостоятельной диагностики; это решение демонстрационной области.

Найти дефект

В Visual Studio откройте MainWindow.xaml.cs CSV-заготовки и нажмите Ctrl+F: ищите CsvStore.Read. Проследите lines[index].Split до cells[4]. Для строки 4 из demo.csv запишите число полученных элементов; обращение к отсутствующему элементу объясняет исключение. Точка останова перед Split и окно «Локальные переменные» покажут фактический массив. После ошибки загрузка списка в DataGrid не выполняется.

Внести исправление

В рабочей копии перед чтением cells[4] добавьте проверку Length == 5. Затем используйте int.TryParse для Id, DateTime.TryParseExact с шаблоном dd.MM.yyyy и InvariantCulture для Due; отдельно проверяйте Item, Owner и Status. Каждую строку обрабатывайте в цикле: для плохой добавляйте «Строка N: причина» и переходите к следующей, для хорошей создавайте Ticket. Возвратите и записи, и предупреждения, чтобы окно могло одновременно показать данные и объяснить пропуск.

Проверка готового изменения

Пересоберите проект, откройте demo.csv. Ожидаются заявки 101 и 102 в таблице, предупреждение о строке 4, внизу «Загружено: 2; пропущено: 1». Закройте предупреждение и убедитесь, что строки остались. Отдельно подмените Id и дату в копии файла и проверьте точность причин.

Результат исправленного импорта

Журнал заявок — WPF версия 1.1

Открыть CSV Экспорт CSV Просрочено

Id	Объект	Ответственный	Срок	Статус
101	Проектор	Ирина	01.01.2025	Открыта
102	Ноутбук	Павел	02.01.2025	Завершена

Предупреждения импорта

Строка 4: ожидается пять полей

OK

Загружено: 2; пропущено: 1

Исправленная демонстрационная сборка: две корректные записи сохранены, строка 4 пропущена с предупреждением.

Две доступные записи

Журнал заявок — WPF версия 1.1

Открыть CSV Экспорт CSV Просрочено

Id	Объект	Ответственный	Срок	Статус
101	Проектор	Ирина	01.01.2025	Открыта
102	Ноутбук	Павел	02.01.2025	Завершена

Загружено: 2; пропущено: 1

После закрытия предупреждения таблица содержит проектор и ноутбук; нижняя строка показывает 2 загруженные и 1 пропущенную запись.

Диагностика D2: совместимость записи и чтения

Материалы для работы

[Скачать заготовку подключения CSV \(ZIP\)](#)

Откройте после первой проверки, чтобы изучить дефекты импорта и экспорта.

[Скачать демонстрационный CSV без дефектной строки \(demo-valid.csv\)](#)

Используйте для отдельной проверки экспорта и повторного импорта.

[Скачать исправленный демонстрационный проект \(ZIP\)](#)

Сверяйтесь после самостоятельной диагностики; это решение демонстрационной области.

Наблюдение

В черновой версии откройте demo-valid.csv, экспортируйте под новым именем и попробуйте снова открыть. Сравните текст файлов. В Visual Studio найдите CsvStore.Write: именно он формирует несовместимый разделитель и дату. Исправляйте в рабочей копии, сохранив исходник и экспорт с дефектом как доказательство.

Правка и контроль

В Write начните список строк с Header и для каждого Ticket соедините пять полей точкой с запятой. Due преобразуйте ToString("dd.MM.yyyy", CultureInfo.InvariantCulture). Перед записью проверьте, что текстовые поля не содержат «;» и перевода строки; этот стенд явно отклоняет такие значения. Соберите, экспортируйте, откройте результат тем же приложением и сравните все поля, а не только количество строк.

Диагностика D3: правило срока

Материалы для работы

[Скачать демонстрационный CSV без дефектной строки \(demo-valid.csv\)](#)

Используйте для отдельной проверки экспорта и повторного импорта.

[Скачать исправленный демонстрационный проект \(ZIP\)](#)

Сверяйтесь после самостоятельной диагностики; это решение демонстрационной области.

Локализация

Откройте demo-valid.csv и нажмите «Просрочено». Найдите CountOverdue в MainWindow.xaml.cs. Выражение с одним сравнением даты ошибочно, если завершённая заявка попадает в результат. Дополните его условием Status != «Завершена» и сравнивайте Due.Date с today.Date.

Граничные случаи

Для проверки создайте временный CSV: открытая вчера, завершённая вчера, открытая сегодня. Ожидается одна просроченная. Затем сделайте срок завтра: счёт не меняется. Верните demo.csv и убедитесь, что исправление не нарушило импорт и предупреждение.

Проверка просрочки

Журнал заявок — WPF версия 1.1

Открыть CSV Экспорт CSV Просрочено

Id	Объект	Ответственный	Срок	Статус
101	Проектор	Ирина	01.01.2025	Открыта
102	Ноутбук	Павел	02.01.2025	Завершена

Просрочено: 1

В исправленной версии завершенная заявка не входит в счётчик: показано «Просрочено: 1».



Подготовка тестов и передача обновления

Матрица

Для каждого случая запишите входной файл, действие, ожидаемое число и значения записей, фактический экран и вывод. Проверьте пустой файл с заголовком, неверный заголовок, отсутствующий файл, неверные Id/дату/статус, пустые поля и ошибочную строку после корректной. Затем выполните цикл импорт → экспорт → импорт в чистой копии и сравните пять полей построчно.

Версия и инструкция

Соберите Release через выбор конфигурации в верхней панели Visual Studio и «Собрать решение». На копии данных проверьте запуск, резервное копирование и откат. В инструкции оператору используйте реальные подписи WPF-кнопок и расшифровку предупреждений. В отчёте свяжите каждое изменение с исходным дефектом и фактической повторной проверкой.

Переход к своему варианту

Материалы для работы

[Скачать исходный проект ServiceDesk \(ZIP\)](#)

Начните с него: соберите и проверьте приложение без CSV.

[Скачать заготовку подключения CSV \(ZIP\)](#)

Откройте после первой проверки, чтобы изучить дефекты импорта и экспорта.

[Скачать исправленный демонстрационный проект \(ZIP\)](#)

Сверяйтесь после самостоятельной диагностики; это решение демонстрационной области.

Изменяемые значения

Вернитесь к проверенной копии стартового WPF-проекта. В MainWindow.xaml поменяйте Title на название предметной области. В MainWindow.xaml.cs найдите TestData.Create: замените демонстрационные Item и Owner на сущности варианта, сохранив структуру Ticket и проверку просрочки. Соберите и повторите первую контрольную точку до подключения CSV.

Стабильный контракт

После этого перенесите CSV-интеграцию и исправления в рабочую копию. Id;Item;Owner;Due;Status, разделитель и формат даты оставьте неизменными; меняются значения Item/Owner и текст инструкции оператору. Откройте только CSV своего варианта. Проверьте две корректные строки, предупреждение о третьей, D1–D3 и особое правило карточки. Исправленный демонстрационный архив помогает сверить общий алгоритм, но не является готовым индивидуальным решением.

Стартовый проект: ServiceDesk.csproj, часть 1

Место и смысл изменения

Откройте файл проекта через обозреватель решений или текстовый редактор. UseWPF включает компиляцию XAML и связывает его с кодом окна.

ServiceDesk.csproj: строки 1–9

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net10.0-windows</TargetFramework>
    <UseWPF>true</UseWPF>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

Проверка законченного участка

Сначала добавьте этот участок в указанный файл и соберите проект. Если компилятор указывает на имя обработчика, проверьте совпадение с Click в XAML. Затем выполните соответствующий сценарий окна и сравните результат с контрольной точкой. В исправленном примере проверьте также demo.csv: две строки, одно предупреждение, просрочена одна заявка.

Стартовый проект: App.xaml, часть 1

Место и смысл изменения

В обозревателе решений откройте App.xaml: это описание точки входа WPF. Сверьте x:Class с классом App в App.xaml.cs.

App.xaml: строки 1–5

```
<Application x:Class="ServiceDeskWpf.App"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
  <Application.Resources />
</Application>
```

Проверка законченного участка

Сначала добавьте этот участок в указанный файл и соберите проект. Если компилятор указывает на имя обработчика, проверьте совпадение с Click в XAML. Затем выполните соответствующий сценарий окна и сравните результат с контрольной точкой. В исправленном примере проверьте также demo.csv: две строки, одно предупреждение, просрочена одна заявка.

Стартовый проект: App.xaml.cs, часть 1

Место и смысл изменения

Откройте вложенный App.xaml.cs. Здесь обработка аргументов демонстрации и самопроверки, затем создание окна. Обычный запуск открывает интерфейс.

App.xaml.cs: строки 1–19

```
using System.Windows;

namespace ServiceDeskWpf;

public partial class App : Application
{
    protected override void OnStartup(StartupEventArgs e)
    {
        base.OnStartup(e);
        if (e.Args.Length == 1 && e.Args[0] == "--self-test")
        {
            var rows = TestData.Create(DateTime.Today);
            Console.WriteLine($"test-records={rows.Count}; overdue={TestData.CountOverdue(rows, DateTime.Today)}");
            Shutdown();
            return;
        }
        new MainWindow(e.Args.Length == 1 && e.Args[0] == "--demo").Show();
    }
}
```

Проверка законченного участка

Сначала добавьте этот участок в указанный файл и соберите проект. Если компилятор указывает на имя обработчика, проверьте совпадение с Click в XAML. Затем выполните соответствующий сценарий окна и сравните результат с контрольной точкой. В исправленном примере проверьте также demo.csv: две строки, одно предупреждение, просрочена одна заявка.

Стартовый проект: MainWindow.xaml, часть 1

Место и смысл изменения

Откройте MainWindow.xaml во вкладке XAML. Здесь расположены Grid, кнопки, таблица с пятью колонками и строка состояния. Для изменения подписи правьте Content, для обработчика — Click.

MainWindow.xaml: строки 1–26

```
<Window x:Class="ServiceDeskWpf.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Журнал заявок – проверка WPF-приложения" Width="960" Height="560">
    <Grid Margin="12">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="*" />
            <RowDefinition Height="Auto" />
        </Grid.RowDefinitions>
        <StackPanel Orientation="Horizontal" Margin="0,0,0,10">
            <Button Content="Тестовые данные" Width="150" Height="32" Margin="0,0,8,0" Click="LoadTestData_Click" />
            <Button Content="Просрочено" Width="140" Height="32" Click="CountOverdue_Click" />
        </StackPanel>
        <DataGrid x:Name="TicketsGrid" Grid.Row="1" AutoGenerateColumns="False" IsReadOnly="True">
            <DataGrid.Columns>
                <DataGridTextColumn Header="Id" Binding="{Binding Id}" />
                <DataGridTextColumn Header="Объект" Binding="{Binding Item}" Width="*" />
                <DataGridTextColumn Header="Ответственный" Binding="{Binding Owner}" Width="*" />
                <DataGridTextColumn Header="Срок" Binding="{Binding Due, StringFormat=dd.MM.yyyy}" Width="130" />
                <DataGridTextColumn Header="Статус" Binding="{Binding Status}" Width="130" />
            </DataGrid.Columns>
        </DataGrid>
        <TextBlock x:Name="StatusText" Grid.Row="2" Margin="0,10,0,0" Text="Сначала загрузите встроенные тестовые данные" />
    </Grid>
</Window>
```



Проверка законченного участка

Сначала добавьте этот участок в указанный файл и соберите проект. Если компилятор указывает на имя обработчика, проверьте совпадение с Click в XAML. Затем выполните соответствующий сценарий окна и сравните результат с контрольной точкой. В исправленном примере проверьте также demo.csv: две строки, одно предупреждение, просрочена одна заявка.

Стартовый проект: MainWindow.xaml.cs, часть 1

Место и смысл изменения

Откройте вложенный MainWindow.xaml.cs. Здесь данные, чтение/запись (в исправленном примере), обработчики и связь с XAML. Вводите код по смысловым участкам ниже, сохраняя порядок строк.

MainWindow.xaml.cs: строки 1–6

```
using System.Windows;

namespace ServiceDeskWpf;

internal record Ticket(int Id, string Item, string Owner, DateTime Due, string Status);
```

Проверка законченного участка

Сначала добавьте этот участок в указанный файл и соберите проект. Если компилятор указывает на имя обработчика, проверьте совпадение с Click в XAML. Затем выполните соответствующий сценарий окна и сравните результат с контрольной точкой. В исправленном примере проверьте также demo.csv: две строки, одно предупреждение, просрочена одна заявка.

Стартовый проект: MainWindow.xaml.cs, часть 2

Место и смысл изменения

Откройте вложенный MainWindow.xaml.cs. Здесь данные, чтение/запись (в исправленном примере), обработчики и связь с XAML. Вводите код по смысловым участкам ниже, сохраняя порядок строк.

MainWindow.xaml.cs: строки 7–18

```
internal static class TestData
{
    public static List<Ticket> Create(DateTime today) =>
    [
        new Ticket(101, "Проектор", "Ирина", today.AddDays(-2), "Открыта"),
        new Ticket(102, "Ноутбук", "Павел", today.AddDays(-1), "Завершена")
    ];

    public static int CountOverdue(IEnumerable<Ticket> tickets, DateTime today) =>
        tickets.Count(t => t.Status != "Завершена" && t.Due.Date < today.Date);
}
```

Проверка законченного участка

Сначала добавьте этот участок в указанный файл и соберите проект. Если компилятор указывает на имя обработчика, проверьте совпадение с Click в XAML. Затем выполните соответствующий сценарий окна и сравните результат с контрольной точкой. В исправленном примере проверьте также demo.csv: две строки, одно предупреждение, просрочена одна заявка.

Стартовый проект: MainWindow.xaml.cs, часть 3

Место и смысл изменения

Откройте вложенный MainWindow.xaml.cs. Здесь данные, чтение/запись (в исправленном примере), обработчики и связь с XAML. Вводите код по смысловым участкам ниже, сохраняя порядок строк.

MainWindow.xaml.cs: строки 19–40

```
public partial class MainWindow : Window
{
    private List<Ticket> tickets = new();

    public MainWindow(bool showTestData = false)
    {
        InitializeComponent();
        if (showTestData) Loaded += (_, _) => LoadTestData();
    }

    private void LoadTestData_Click(object sender, RoutedEventArgs e) => LoadTestData();

    private void LoadTestData()
    {
        tickets = TestData.Create(DateTime.Today);
        TicketsGrid.ItemsSource = tickets.ToList();
        StatusText.Text = $"Тестовых записей: {tickets.Count}";
    }

    private void CountOverdue_Click(object sender, RoutedEventArgs e) =>
        StatusText.Text = $"Просрочено: {TestData.CountOverdue(tickets, DateTime.Today)}";
}
```



Проверка законченного участка

Сначала добавьте этот участок в указанный файл и соберите проект. Если компилятор указывает на имя обработчика, проверьте совпадение с Click в XAML. Затем выполните соответствующий сценарий окна и сравните результат с контрольной точкой. В исправленном примере проверьте также demo.csv: две строки, одно предупреждение, просрочена одна заявка.

Исправленный пример: ServiceDesk-fixed.csproj, часть 1

Место и смысл изменения

Откройте файл проекта через обозреватель решений или текстовый редактор. UseWPF включает компиляцию XAML и связывает его с кодом окна.

ServiceDesk-fixed.csproj: строки 1–10

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net10.0-windows</TargetFramework>
    <UseWPF>true</UseWPF>
    <RootNamespace>ServiceDeskWpf</RootNamespace>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

Проверка законченного участка

Сначала добавьте этот участок в указанный файл и соберите проект. Если компилятор указывает на имя обработчика, проверьте совпадение с Click в XAML. Затем выполните соответствующий сценарий окна и сравните результат с контрольной точкой. В исправленном примере проверьте также demo.csv: две строки, одно предупреждение, просрочена одна заявка.

Исправленный пример: App.xaml, часть 1

Место и смысл изменения

В обозревателе решений откройте App.xaml: это описание точки входа WPF. Сверьте x:Class с классом App в App.xaml.cs.

App.xaml: строки 1–5

```
<Application x:Class="ServiceDeskWpf.App"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
  <Application.Resources />
</Application>
```

Проверка законченного участка

Сначала добавьте этот участок в указанный файл и соберите проект. Если компилятор указывает на имя обработчика, проверьте совпадение с Click в XAML. Затем выполните соответствующий сценарий окна и сравните результат с контрольной точкой. В исправленном примере проверьте также demo.csv: две строки, одно предупреждение, просрочена одна заявка.

Исправленный пример: App.xaml.cs, часть 1

Место и смысл изменения

Откройте вложенный App.xaml.cs. Здесь обработка аргументов демонстрации и самопроверки, затем создание окна. Обычный запуск открывает интерфейс.

App.xaml.cs: строки 1–30

```
using System.IO;
using System.Windows;

namespace ServiceDeskWpf;

public partial class App : Application
{
    protected override void OnStartup(StartupEventArgs e)
    {
        base.OnStartup(e);
        if (e.Args.Length == 2 && e.Args[0] == "--self-test")
        {
            var result = CsvStore.Read(e.Args[1]);
            var target = Path.GetTempFileName();
            try
            {
                CsvStore.Write(target, result.Tickets);
                var again = CsvStore.Read(target);
                if (again.Warnings.Count != 0 || !result.Tickets.SequenceEqual(again.Tickets))
                    throw new InvalidOperationException("Повторный импорт не совпал");
                Console.WriteLine($"tickets={result.Tickets.Count}; warnings={result.Warnings.Count}; overdue={CsvStore.CountOverdue(result.Tickets,
DateTime.Today)}");
                foreach (var warning in result.Warnings) Console.WriteLine(warning);
            }
            finally { File.Delete(target); }
            Shutdown();
            return;
        }
    }
}
```



```
        new MainWindow(e.Args.Length == 2 && e.Args[0] == "--demo" ? e.Args[1] : null).Show();
    }
}
```

Проверка законченного участка

Сначала добавьте этот участок в указанный файл и соберите проект. Если компилятор указывает на имя обработчика, проверьте совпадение с Click в XAML. Затем выполните соответствующий сценарий окна и сравните результат с контрольной точкой. В исправленном примере проверьте также demo.csv: две строки, одно предупреждение, просрочена одна заявка.

Исправленный пример: MainWindow.xaml, часть 1

Место и смысл изменения

Откройте MainWindow.xaml во вкладке XAML. Здесь расположены Grid, кнопки, таблица с пятью колонками и строка состояния. Для изменения подписи правьте Content, для обработчика — Click.

MainWindow.xaml: строки 1–27

```
<Window x:Class="ServiceDeskWpf.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Журнал заявок – WPF версия 1.1" Width="960" Height="560">
    <Grid Margin="12">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="*" />
            <RowDefinition Height="Auto" />
        </Grid.RowDefinitions>
        <StackPanel Orientation="Horizontal" Margin="0,0,0,10">
            <Button Content="Открыть CSV" Width="140" Height="32" Margin="0,0,8,0" Click="OpenCsv_Click" />
            <Button Content="Экспорт CSV" Width="140" Height="32" Margin="0,0,8,0" Click="SaveCsv_Click" />
            <Button Content="Просрочено" Width="140" Height="32" Click="CountOverdue_Click" />
        </StackPanel>
        <DataGrid x:Name="TicketsGrid" Grid.Row="1" AutoGenerateColumns="False" IsReadOnly="True">
            <DataGrid.Columns>
                <DataGridTextColumn Header="Id" Binding="{Binding Id}" />
                <DataGridTextColumn Header="Объект" Binding="{Binding Item}" Width="*" />
                <DataGridTextColumn Header="Ответственный" Binding="{Binding Owner}" Width="*" />
                <DataGridTextColumn Header="Срок" Binding="{Binding Due, StringFormat=dd.MM.yyyy}" Width="130" />
                <DataGridTextColumn Header="Статус" Binding="{Binding Status}" Width="130" />
            </DataGrid.Columns>
        </DataGrid>
        <TextBlock x:Name="StatusText" Grid.Row="2" Margin="0,10,0,0" Text="Откройте CSV-файл" />
    </Grid>
</Window>
```



Проверка законченного участка

Сначала добавьте этот участок в указанный файл и соберите проект. Если компилятор указывает на имя обработчика, проверьте совпадение с Click в XAML. Затем выполните соответствующий сценарий окна и сравните результат с контрольной точкой. В исправленном примере проверьте также demo.csv: две строки, одно предупреждение, просрочена одна заявка.

Исправленный пример: MainWindow.xaml.cs, часть 1

Место и смысл изменения

Откройте вложенный MainWindow.xaml.cs. Здесь данные, чтение/запись (в исправленном примере), обработчики и связь с XAML. Вводите код по смысловым участкам ниже, сохраняя порядок строк.

MainWindow.xaml.cs: строки 1–11

```
using System.Globalization;
using System.IO;
using System.Text;
using System.Windows;
using Microsoft.Win32;

namespace ServiceDeskWpf;

internal record Ticket(int Id, string Item, string Owner, DateTime Due, string Status);
internal record ReadResult(List<Ticket> Tickets, List<string> Warnings);
```

Проверка законченного участка

Сначала добавьте этот участок в указанный файл и соберите проект. Если компилятор указывает на имя обработчика, проверьте совпадение с Click в XAML. Затем выполните соответствующий сценарий окна и сравните результат с контрольной точкой. В исправленном примере проверьте также demo.csv: две строки, одно предупреждение, просрочена одна заявка.

Исправленный пример: MainWindow.xaml.cs, часть 2

Место и смысл изменения

Откройте вложенный MainWindow.xaml.cs. Здесь данные, чтение/запись (в исправленном примере), обработчики и связь с XAML. Вводите код по смысловым участкам ниже, сохраняя порядок строк.

MainWindow.xaml.cs: строки 12–55

```
internal static class CsvStore
{
    private const string Header = "Id;Item;Owner;Due;Status";

    public static ReadResult Read(string path)
    {
        var tickets = new List<Ticket>();
        var warnings = new List<string>();
        var lines = File.ReadAllLines(path, Encoding.UTF8);
        if (lines.Length == 0 || lines[0].TrimStart('\uFEFF') != Header)
            throw new InvalidDataException($"Ожидается заголовок {Header}");
        for (int index = 1; index < lines.Length; index++)
        {
            if (string.IsNullOrEmpty(lines[index])) continue;
            var cells = lines[index].Split(';');
            string? reason = null;
            if (cells.Length != 5) reason = "ождается пять полей";
            else if (!int.TryParse(cells[0], out _)) reason = "неверный Id";
            else if (string.IsNullOrEmpty(cells[1]) || string.IsNullOrEmpty(cells[2])) reason = "пустое обязательное поле";
            else if (!DateTime.TryParseExact(cells[3], "dd.MM.yyyy", CultureInfo.InvariantCulture, DateTimeStyles.None, out _)) reason = "неверная да
та";
            else if (cells[4] != "Открыта" && cells[4] != "Завершена") reason = "неверный статус";
            if (reason != null) { warnings.Add($"Строка {index + 1}: {reason}"); continue; }
            tickets.Add(new Ticket(int.Parse(cells[0], CultureInfo.InvariantCulture), cells[1], cells[2],
                DateTime.ParseExact(cells[3], "dd.MM.yyyy", CultureInfo.InvariantCulture), cells[4]));
        }
        return new ReadResult(tickets, warnings);
    }
}
```

```
public static void Write(string path, IEnumerable<Ticket> tickets)
{
    var lines = new List<string> { Header };
    foreach (var t in tickets)
    {
        if (new[] { t.Item, t.Owner, t.Status }.Any(value => value.Contains(';') || value.Contains('\n') || value.Contains('\r')))
            throw new InvalidDataException("В поле нельзя использовать разделитель или перевод строки");
        lines.Add($"{t.Id};{t.Item};{t.Owner};{t.Due.ToString("dd.MM.yyyy", CultureInfo.InvariantCulture)};{t.Status}");
    }
    File.WriteAllLines(path, lines, Encoding.UTF8);
}

public static int CountOverdue(IEnumerable<Ticket> tickets, DateTime today) =>
    tickets.Count(t => t.Status != "Завершена" && t.Due.Date < today.Date);
}
```

Проверка законченного участка

Сначала добавьте этот участок в указанный файл и соберите проект. Если компилятор указывает на имя обработчика, проверьте совпадение с Click в XAML. Затем выполните соответствующий сценарий окна и сравните результат с контрольной точкой. В исправленном примере проверьте также demo.csv: две строки, одно предупреждение, просрочена одна заявка.

Исправленный пример: MainWindow.xaml.cs, часть 3

Место и смысл изменения

Откройте вложенный MainWindow.xaml.cs. Здесь данные, чтение/запись (в исправленном примере), обработчики и связь с XAML. Вводите код по смысловым участкам ниже, сохраняя порядок строк.

MainWindow.xaml.cs: строки 56–98

```
public partial class MainWindow : Window
{
    private List<Ticket> tickets = new();

    public MainWindow(string? demoPath = null)
    {
        InitializeComponent();
        if (demoPath != null) Loaded += (_, _) =>
            Dispatcher.BeginInvoke(System.Windows.Threading.DispatcherPriority.ApplicationIdle,
                new Action(() => LoadCsv(demoPath)));
    }

    private void OpenCsv_Click(object sender, RoutedEventArgs e)
    {
        var dialog = new OpenFileDialog { Filter = "CSV (*.csv)|*.csv" };
        if (dialog.ShowDialog() == true) LoadCsv(dialog.FileName);
    }

    private void LoadCsv(string path)
    {
        try
        {
            var result = CsvStore.Read(path);
            tickets = result.Tickets;
            TicketsGrid.ItemsSource = tickets.ToList();
            StatusText.Text = $"Загружено: {tickets.Count}; пропущено: {result.Warnings.Count}";
            if (result.Warnings.Count > 0)
                MessageBox.Show(string.Join(Environment.NewLine, result.Warnings), "Предупреждения импорта");
        }
    }
}
```

```
    }  
    catch (Exception ex) { MessageBox.Show(ex.Message, "Ошибка импорта"); }  
}  
  
private void SaveCsv_Click(object sender, RoutedEventArgs e)  
{  
    var dialog = new SaveFileDialog { Filter = "CSV (*.csv)|*.csv", FileName = "export.csv" };  
    if (dialog.ShowDialog() != true) return;  
    try { CsvStore.Write(dialog.FileName, tickets); StatusText.Text = "Файл сохранён"; }  
    catch (Exception ex) { MessageBox.Show(ex.Message, "Ошибка экспорта"); }  
}  
  
private void CountOverdue_Click(object sender, RoutedEventArgs e) =>  
    StatusText.Text = $"Просрочено: {CsvStore.CountOverdue(tickets, DateTime.Today)}";  
}
```

Проверка законченного участка

Сначала добавьте этот участок в указанный файл и соберите проект. Если компилятор указывает на имя обработчика, проверьте совпадение с Click в XAML. Затем выполните соответствующий сценарий окна и сравните результат с контрольной точкой. В исправленном примере проверьте также demo.csv: две строки, одно предупреждение, просрочена одна заявка.